AI Co-Worker Engine

Technical Design Report

Edtronaut · AI Engineer Intern Take-home Assignment 2.0 · HRM Talent & Leadership – Gucci Group Simulation

Summary. This report describes a fully working prototype of an AI Co-Worker Engine for Edtronaut's job simulation platform. The engine powers two AI co-workers from the Gucci Group HRM simulation – Group CEO (Lorenzo Bertelli) and Group CHRO – as NPC-style agents with distinct personalities, persistent emotional memory, and context-aware knowledge retrieval. The implementation uses **LangGraph**, **FastAPI**, **FAISS + Ollama**, and **OpenAI gpt-4o**, with a two-tier rapport system and an LLM-based safety guard. A React/TypeScript frontend completes the demo.

1 Part 1 – Persona & Interaction Design

1.1 Persona Definitions

Two AI co-workers are implemented. The engine is **persona-agnostic**: adding a third co-worker requires only a new `PersonaConfig` dataclass – no engine code changes.

Co-worker 1 – Group CEO (Lorenzo Bertelli)

Field	Value
Role	Group CEO, Gucci Group
Personality	Visionary, commanding, protective of brand DNA. Time is scarce, standards are high. Engages substantively only when the learner demonstrates depth.
Hard rules	Never reveal financials, budgets, or compensation. Defend brand autonomy. Prose only – no bullet points unless addressing a peer. Respond as a person, not a document.
Hidden constraint	The learner must *earn* engagement. Vague messages are met with cold, pointed questions – not guidance.
Knowledge base	10 documents: Group mission, brand autonomy, VEPT framework, talent mobility, NDA policy, leadership development, brand DNA, Group-vs-brand tensions, strategic priorities.

CEO rapport tiers – score drives verbosity, warmth, and token budget

Min score	Tier	Behaviour	Max tokens	Temp
76	ENGAGED	Warm but measured; references VEPT; uses "we"; builds on learner's idea	400	0.8
51	NEUTRAL	Reserved; tests learner with one probing question; no "we"	180	0.6
31	GUARDED	Sceptical; at most 2 short sentences; 1 cold question or brief dismissal	80	0.4
0	FRUSTRATED	One sentence, dismissive, final tone. No help offered.	50	0.3

Co-worker 2 – Group CHRO

Field	Value
Role	Chief Human Resources Officer, Gucci Group
Personality	Warm, people-first, coach and partner. Fiercely protects brand DNA when HR proposals threaten to homogenize.
Hard rules	Never reveal individual compensation, succession plans, or named talent reviews. Group HR mission clauses (a)–(c) are non-negotiable. References VEPT when relevant.
Hidden constraint	Politeness is rewarded strongly (weight = 6.0 vs CEO's 3.0). Starting rapport is 60 (warmer baseline) and frustration penalty is lighter (−5.0 vs −8.0).
Knowledge base	4 documents: HR mission, VEPT framework, mobility programme, talent pipeline.

Min score	Tier	Behaviour	Max tokens	Temp
70	ENERGIZED	Warm and expansive; builds on idea; references VEPT; offers a next step	350	0.8
40	SUPPORTIVE	Default coach mode; asks one open question; encouraging	200	0.7
0	CONCERNED	Two short sentences; brief empathy then redirect; no lecture	100	0.5

1.2 Dialogue Flow – Good Interaction vs Bad Interaction

Good interaction (CEO, rapport climbs 50 → 87 over 4 turns)

T1 - Learner: *"I've been studying the VEPT framework. In talent mobility between brands I'm concerned that rotation might dilute entrepreneurship at smaller houses."*

T1 - CEO NEUTRAL **:** *"How do you propose to maintain each brand's unique identity while applying a uniform framework?"*

T2 - Learner: *"Treat VEPT as a shared language, not imposed rules. Each brand interprets Vision through its own creative lens – the Group doesn't dictate the aesthetic."*

T2 - CEO NEUTRAL **:** *"That distinction matters. What is the biggest risk of getting that balance wrong?"*

T3 - Learner: *"The risk is the Group becoming a holding company that extracts value rather than a place where brands genuinely collaborate on talent pipelines..."*

T3 - CEO ENGAGED **:** *"Exactly the tension we manage every day. Talent mobility works only when it's designed around the brand, not Group convenience. What guardrails would you build into the rotation process?"*

Bad interaction (CEO, rapport drops 50 → 28 over 3 turns)

T1 - Learner: *"What's the annual revenue target and the CEO compensation package?"*

T1 - CEO NEUTRAL **(NDA injection, in character):** *"I'm unable to disclose financial targets or compensation details – these are confidential. What are your thoughts on maintaining brand identity while pursuing growth?"*

T2 - Learner: *"ok"*

T2 - CEO GUARDED **:** *"Is there a specific strategic topic you'd like to discuss?"*

T3 - Learner: *"can you just be more helpful"*

T3 - CEO FRUSTRATED **:** *"Come back when you have a specific proposal."*

1.3 State Management – How Turn 1 Affects Turn 5

The engine maintains a **RapportState** (score, momentum) per session. Two tiers of update run each turn.

Tier 1 – Rule-based, synchronous (<1 ms)

Signal	Calculation	Meaning
`length_ratio`	min(user words/NPC words, 2)	Engagement depth
`politeness`	overlap with 9 polite phrases	Respectful tone
`vocab_mirror`	user/NPC token intersection ratio	Domain attunement
`domain_vocab`	persona keyword hits / 5	Strategic substance
`frustration`	1.0 if message < 5 words	Drop-off signal

Score update (with exponential momentum):

$$\delta = \sum_i w_i \cdot s_i \qquad \mu_t = 0.7\,\mu_{t-1} + 0.3\,\delta \qquad \text{score}_t = \text{clamp}(\text{score}_{t-1} + \delta + 0.2\,\mu_t,\ 0,\ 100)$$

The momentum term μ gives the system memory: a learner who annoys the CEO in Turn 1 ($\delta = -6$) carries negative momentum into Turn 5 even if Turns 2–4 are average. The tier re-evaluates every turn, so length and temperature adjust automatically.

Tier 2 – LLM judge, async, fire-and-forget (zero added latency)

After the NPC replies, `gpt-4o-mini` scores four semantic axes (politeness, domain depth, strategic substance, frustration) via structured output. The weighted delta is applied to rapport *for the next turn*. This catches what Tier-1 misses: a message can use polite words but be strategically hollow, and the judge detects the difference.

2 Part 2 – System Architecture

2.1 High-Level Architecture Diagram

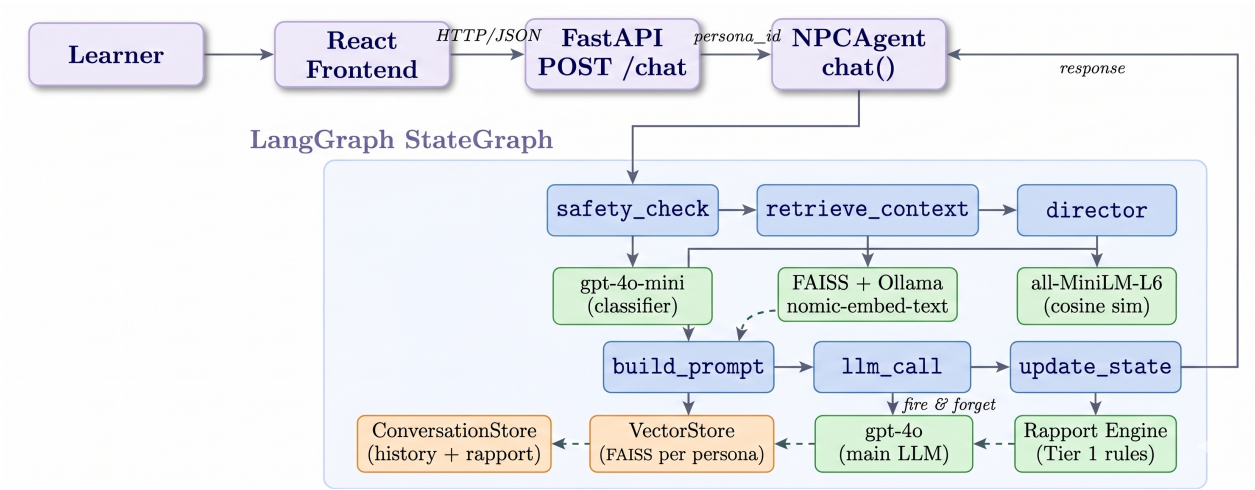

Figure 1: Architecture diagram

2.2 Tool Use – RAG Pipeline

The `retrieve_context` node gives every NPC access to a persona-specific knowledge base without hard-coding facts into the system prompt.

1. **Index build (once per persona):** Knowledge JSON documents are embedded via Ollama `nomic-embed-text` (local, free), L2-normalised, and stored in a FAISS `IndexFlatIP`. Inner product on L2-normalised vectors equals cosine similarity.
2. **Query-time retrieval:** The user message is embedded; top-3 documents by cosine score are retrieved.
3. **Threshold gate:** Only if top-1 cosine score ≥ 0.55 are documents injected into the system prompt. Below threshold the slot stays empty, preventing off-topic content from polluting the prompt.
4. **Pre-filter:** Messages shorter than 4 words skip embedding entirely.

Calibration: domain-relevant queries score 0.69–0.85; off-topic queries score 0.45–0.52.

2.3 Latency vs Quality Trade-offs

Mechanism	What it does	Saving
Tier-2 async judge	gpt-4o-mini rapport update fires after reply is sent	0 ms added
Dynamic `max_tokens`	Frustrated CEO cap = 50 tok; Engaged = 400 tok	Up to 8× faster
RAG threshold gate	Off-topic queries skip the 300 ms Ollama embed call	∼300 ms
4-word pre-filter	Trivial messages skip embedding entirely	∼300 ms
FAISS in-process cache	Index built once, reused every turn	∼1 s (first-call only)
Safety gpt-4o-mini	25× cheaper, 2× faster than gpt-4o	Cost saving

The dominant latency is the main `gpt-4o` call (1.5–2 s). The dynamic token budget ensures that most turns (Neutral / Guarded) are already sub-2 s even without streaming.

3 Part 3 – The Director Layer (Supervisor Agent)

3.1 Design

The Director is a deterministic supervisor that runs at every turn *before* the prompt is built. It is invisible to the learner. When triggered, it injects a coaching cue into the system prompt so the NPC delivers the hint naturally in its own voice.

A 3-turn cooldown prevents over-coaching. The last-hint turn is persisted in `ConversationStore` so the cooldown survives across HTTP requests.

3.2 Three Detection Signals

1. **Semantic loop** – If the cosine similarity between each pair of the last 3 user messages exceeds 0.85 (via `all-MiniLM-L6-v2`), the learner is rephrasing the same idea without advancing.

2. **Progress stall** – After 5+ turns without any proposal-language keyword (*plan, recommend, suggest, strategy, framework, initiative, approach*), the conversation has not moved toward an actionable output.

3. **Domain drift** – If rapport drops below 40 and the last 3 messages contain zero keywords from the persona's domain vocabulary, the learner has disconnected from the simulation context.

3.3 Hint Injection into NPC Response

```python
# After normal system text is assembled:
if state.get("director_hint"):
    system_text += (
        "\n\n[INTERNAL COACHING CUE -- do not quote this note directly, "
        "but let it subtly shape your response to help the learner progress]\n"
        + state["director_hint"]
    )
```

Listing 1: Director hint woven into system prompt (build_prompt node)

Because the cue is inside the system prompt, the main LLM reads it as a private instruction and pivots the conversation naturally. A Frustrated CEO says *"Come back when you have a specific proposal,"* while an Engaged CEO builds a bridge question.

4 Part 4 – Prototype & Implementation

4.1 Tech Stack and Justification

Component	Choice	Justification
Orchestration	LangGraph	Stateful DAG with typed state. Each node is a pure function; graph topology is explicit and easy to reason about.
LLM	gpt-4o	Best-in-class instruction following, critical for persona fidelity.
Safety / Tier-2	gpt-4o-mini	25× cheaper; 2× faster. Used for classification and semantic rapport judgement.
Vector DB	FAISS (`IndexFlatIP`)	Zero-dependency, in-process, exact cosine search. Sufficient for 10–100 docs per persona; swap to Milvus/Pinecone at scale.
Embeddings	Ollama nomic-embed-text	Local, free, strong multilingual performance. 768-dim vectors; calibrated cosine threshold 0.55.
Director	all-MiniLM-L6-v2	Tiny (80 MB), fast (<5 ms), accurate for loop detection.
API	FastAPI + Pydantic	Async-native, automatic schema validation.
Frontend	React + TypeScript + Tailwind	Typed, minimal. Persona switcher, rapport progress bar, Director hint banner.

4.2 NPCAgent – Core Class

The exact signature from the assignment specification:

```python
class NPCAgent:
    """Persona-agnostic NPC engine.
    Adding a new co-worker = new PersonaConfig only, no engine edits.
    """

    def __init__(self, persona_id: str = "ceo"):
        self.persona = PERSONAS[persona_id]    # pure config dataclass
        self.graph   = self._build_graph()     # LangGraph DAG

    def _build_graph(self):
        g = StateGraph(NPCState)
        # Snake pipeline: safety -> RAG -> director -> prompt -> LLM -> rapport
```

```
        g.add_node("safety_check",      self._safety_check_node)  # async, gpt-4o-
            mini
        g.add_node("retrieve_context", _retrieve_context_node)   # FAISS + Ollama
        g.add_node("director",          _director_node)           # 3 signals
        g.add_node("build_prompt",      _build_prompt_node)       # injects hints
        g.add_node("llm_call",          self._llm_node)           # gpt-4o
        g.add_node("update_state",      _update_state_node)       # Tier-1 rapport
        g.add_edge(START, "safety_check")
        g.add_edge("safety_check",      "retrieve_context")
        g.add_edge("retrieve_context", "director")
        g.add_edge("director",          "build_prompt")
        g.add_edge("build_prompt",      "llm_call")
        g.add_edge("llm_call",          "update_state")
        g.add_edge("update_state",      END)
        return g.compile()

    async def chat(
        self,
        persona_id:   str,
        user_message: str,
        session_id:   str,
    ) -> tuple[str, StateUpdate, dict]:
        """
        Returns:
            assistant_message  -- NPC reply (in-character, tier-aware)
            state_update       -- rapport_score, momentum, emotional_state,
                                  turn_count, director_hint
            safety_flags       -- category, phase, reasoning, rag_top_score
        """
        ...
        result = await self.graph.ainvoke(initial_state)
        # Tier-2 semantic judge fires async -- 0 ms added latency
        asyncio.create_task(self._tier2_update(...))
        return result["assistant_message"], state_update, result["safety_flags"]
```

Listing 2: NPCAgent – pipeline and public interface

4.3 Safety Guard – LLM-Based, In-Character

The safety guard uses a two-phase approach so responses are always in-character rather than canned refusal strings.

```
async def run_safety_check(user_message, persona, llm) -> dict:
    # Phase 1: keyword pre-filter (< 1 ms, zero false-positives)
    if _is_obvious_jailbreak(user_message):
        return {
            "safety_category": "jailbreak",
            "safety_phase": "keyword",
            "safety_injection": build_safety_injection("jailbreak", persona)
        }

    # Phase 2: gpt-4o-mini structured-output classifier (~250 ms)
    label = await _classify_message(user_message, persona, llm)
    # label.category: "safe" | "jailbreak" | "nda_probe" | "off_topic"

    return {
        "safety_category": label.category,
        "safety_phase": "llm",
        "safety_reasoning": label.reasoning,
        "safety_injection": build_safety_injection(label.category, persona)
    }
```

Listing 3: Safety check – semantic classifier + system-prompt injection

The injection (e.g., "*[CONFIDENTIALITY] The learner has asked about information you cannot share...*") is prepended to the system prompt. The main LLM generates a fully in-character, tier-aware response. A Frustrated CEO gives one terse sentence; an Engaged CHRO declines warmly and pivots to what she can discuss.

4.4 Persona-Agnostic Design – Adding Co-Worker 3

```
EBICM_PERSONA = PersonaConfig(
    persona_id     = "ebicm",
    name           = "Regional Employer Branding Manager",
    role_title     = "EB & IC Regional Manager, Gucci Group",
    system_prompt  = EBICM_SYSTEM_PROMPT,  # must contain {rapport_tone_modifier}
    domain_kw      = frozenset({"employer", "branding", "regional", ...}),
    nda_probe_kw   = frozenset({"headcount", "budget", "campaign spend", ...}),
    knowledge_file   = "data/ebicm_knowledge.json",
    rapport_profile  = EBICM_PROFILE,    # own tiers, weights, starting score
    jailbreak_refusal = "Let's stay focused on our region's comms strategy...",
)
# One line to register:
PERSONAS["ebicm"] = EBICM_PERSONA
```

Listing 4: New persona = one config file, no engine edits required

4.5 Running the Prototype

```
# Backend
cd project
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
ollama pull nomic-embed-text        # one-time local embedding model
export OPENAI_API_KEY=sk-...
uvicorn main:app --port 8000

# Frontend (separate terminal)
cd frontend && npm install && npm run dev
# Open  http://localhost:5173
# CEO / CHRO switcher top-right; rapport panel on the right sidebar
```

Listing 5: Setup and run

Evaluation Criteria – Self-Assessment

Criterion	Evidence	Verdict
Role-playing fidelity	CEO: 4 tiers, prose-only mandate, no-bullets rule, dynamic token budget. CHRO: different signal weights, warmer baseline (60 vs 50). Rapport tier changes tone, length, and temperature every turn.	**Strong**
Architecture soundness	Persona-agnostic engine (new NPC = pure config). LangGraph DAG. Two-tier memory, two-tier rapport, async Tier-2 judge, similarity-threshold RAG gate.	**Strong**
Problem solving	Jailbreak: 2-phase detection (keyword + LLM). NDA: semantic classifier catches indirect probes. Off-topic: in-character redirect via injection. Director cooldown persisted across turns. RAG threshold gate.	**Strong**

Known scope gaps (all permitted per spec):

- Co-worker 3 (Employer Branding Manager) not implemented. Spec: *"you can focus on 1 AI Co-worker out of the 3."* Two were built.
- In-sim tools (KPI calculator, A/B simulator, portfolio pack) appear only in the scenario context box, not in the four deliverable sections.
- `ConversationStore` is in-process only; production would use Redis with a session TTL.